

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KODEHAWK - A HYBRID AI-BASED FRAMEWORK FOR DETECTING CODE SMELLS, VULNERABILITIES AND PERFORMANCE ISSUES

Dr. P. Kumar
Professor

Shanthosh S - 220701263
Vignesh C - 220701317

INTRODUCTION

- Our project operates in the field of **Automated Code Analysis**. The rise of AI-powered tools like GitHub Copilot is revolutionizing software development, enabling even beginners to build complex applications.
- A critical issue has emerged where developers, especially those with less experience, often blindly trust the code automatically generated by these AI assistants without proper verification.
- This unverified trust can lead to severe security vulnerabilities, such as exposed API keys and data breaches, sometimes causing entire systems to fail, as seen in real-world incidents.
- Beyond security, **AI-generated code can suffer from poor quality** ("code smells") that makes it difficult to maintain and can contain inefficient logic that leads to slow performance and a poor user experience.

Therefore, there is an urgent need for a system like KodeHawk to act as a post-generation safety net, automatically auditing AI-assisted code to identify these hidden risks and ensure the final product is secure, reliable, and high-quality.

PROJECT OVERVIEW

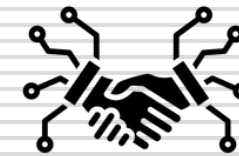
- The primary goal of KodeHawk is to serve as an intelligent and automated "safety net" for modern software development, particularly for codebases that heavily utilize AI-assisted coding tools. Our project aims to bridge the gap between the rapid code generation enabled by AI and the critical, often overlooked, need for comprehensive code verification.
- We also aim to achieve the following



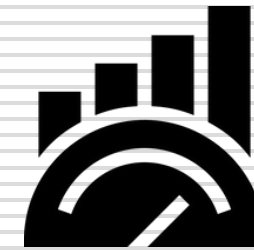
**Automate the
Detection of
Hidden Risks**



**Provide
Actionable
Insights**



**Build a Layer of
Trust for AI-
Generated Code**



**Establish a
Benchmark for AI
Code Quality**

PROJECT OVERVIEW

Overall Approach

- To create a **Hybrid AI-Based Framework** that combines the strengths of traditional, rule-based analysis with the contextual understanding of modern **Large Language Models (LLMs)**.
- Traditional tools are fast and precise but lack an understanding of nuance and intent, while LLMs excel at understanding context but can sometimes miss specific, rule-based errors.
- Our approach is built on a sophisticated **Multi-Agent System Architecture**
 - **The Hybrid Engine:**
 - **Traditional Static Analysis:** We integrate proven, industry-standard tools like SonarQube and Abstract Syntax Tree (AST)-based analysis. These tools form the first line of defense, efficiently catching well-defined, common coding errors and vulnerabilities based on established rulesets.
 - **Large Language Model (LLM) Intelligence:** We leverage the power of LLMs to perform a deeper, more semantic analysis of the code. The LLMs can understand the developer's intent, evaluate design patterns, and identify complex logical flaws that static analyzers would typically miss.

EXISTING SYSTEM

- **Traditional Static Analysis (SAST) Tools:**

- These are the established industry-standard tools for code quality and security analysis. Examples include SonarQube, Checkmarx, and Veracode.
- They **operate on a rule-based engine**. They parse the source code into a structured format, like an Abstract Syntax Tree (AST), and then run it against a predefined set of rules to find patterns that match known vulnerabilities, bugs, or "code smells." For example, a rule might look for SQL query strings that are concatenated with user input, a classic sign of a potential SQL injection vulnerability.
- They are **excellent at catching well-defined, common coding errors** and enforcing consistent coding standards across large teams.

- **Emerging LLM-Based Analysis Tools:**

- This is a newer category of tools that **leverage the power of Large Language Models (LLMs)**—the same technology behind ChatGPT and GitHub Copilot—to analyze code.
- Instead of rigid rules, these tools have a **semantic understanding of the code**. They can comprehend the developer's intent, analyze the logical flow, and assess the context in which the code operates. This allows them to identify more complex and nuanced issues that rule-based systems would miss.
- They are used for deeper code understanding, generating documentation, suggesting refactoring options, and identifying complex logical flaws.

EXISTING SYSTEM

- **Limitations of Traditional SAST Tools:**

- These tools are notorious for **flagging issues that are not actual problems**. Developers can become desensitized to their warnings, leading to "alert fatigue" where even critical alerts are ignored.
- A **rule-based system cannot understand the intent behind the code**. It might flag a piece of code as a vulnerability when, in that specific context, it is perfectly safe. It doesn't understand the "why," only the "what."
- Traditional tools are limited to their predefined rule sets. They cannot detect novel or complex vulnerabilities that involve multiple, interacting parts of the application. They are blind to business logic flaws or subtle design pattern errors.

- **Limitations of LLM-Based Tools:**

- LLMs can sometimes be confidently incorrect. They **might "hallucinate" a vulnerability** that doesn't exist or provide a flawed recommendation for a fix, which can mislead developers.
- Running a **deep semantic analysis on an entire enterprise-level codebase is computationally expensive and time-consuming**. This makes it challenging to integrate pure LLM analysis into rapid, real-time development cycles (like CI/CD pipelines).
- The output of an LLM can vary even with the same input. For compliance and security auditing, this lack of consistent, repeatable analysis can be a significant drawback.

PROBLEM STATEMENT

- The core problem is **the unverified and often blind trust placed in the code generated by AI-assisted coding tools**. The rapid adoption of platforms like GitHub Copilot has enabled developers, especially novices and hobbyists, to build and deploy applications without a deep understanding of the underlying logic. This creates a critical gap where code is moved into production without undergoing the necessary scrutiny for quality, security, and performance.
- Consequently, **modern codebases are increasingly susceptible to containing hidden flaws introduced by AI, such as critical security vulnerabilities, subtle performance bottlenecks, and poor architectural practices ("code smells")**. Existing analysis tools are ill-equipped to handle this new paradigm, as traditional static analyzers lack the contextual understanding to identify complex logical flaws, while pure LLM-based tools are often too slow and unreliable for systematic, large-scale verification.

OBJECTIVES AND SCOPE

Key objectives of your project:

- **To Develop a Hybrid Analysis Engine:** The primary objective is to engineer a sophisticated framework that synergizes the strengths of traditional static analysis (using tools like SonarQube and AST parsing) with the deep contextual understanding of Large Language Models (LLMs).
- **To Implement a Collaborative Multi-Agent System:** To construct a multi-agent architecture using LangGraph. This involves creating specialized agents—an Orchestrator, Analyser, Tester, and Architect—that work in concert to perform a holistic, multi-faceted review of the codebase.
- **To Deliver Actionable Intelligence:** A key goal is to go beyond simple error flagging. The project will focus on generating a single, consolidated report that not only lists the identified issues but also provides clear explanations and actionable recommendations for remediation, empowering developers to improve their code.

OBJECTIVES AND SCOPE

What is included in the scope

- **Input Method:** Public GitHub repository URLs
- **Core Technologies:** Backend using FastAPI and the core logic will be built using Langgraph
- **Analysis Focus:** The analysis will specifically target a curated set of high-impact issues:
 - Code Smells, Vulnerabilities, Performance
- **Supported Languages:** The initial implementation will focus on one or two popular languages to ensure the depth and accuracy of the analysis.
- **Final Output:** A detailed report presented to the user.

What is excluded in the scope

- **Automated Code Correction:** KodeHawk is a detection and recommendation tool. It will not automatically refactor or fix the identified issues in the source code.
- **Real-time IDE Integration:** This version will not be a real-time plugin for IDEs like VS Code.
- **Private Repositories:** The analysis of private GitHub repositories is not included in the current scope.
- **Dynamic Analysis (DAST):** The project is focused entirely on Static Analysis (SAST)—analyzing the code at rest.

LITERATURE SURVEY

Author(s) / Year	Title of the Paper	Methodology / Approach	Inference
Pearce et al. (2022)	Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions.	Performed a study on code generated by GitHub Copilot across 89 different scenarios, covering top CWEs (Common Weakness Enumerations).	Found that in multiple languages, approximately 40% of the code generated by Copilot contained security vulnerabilities. This directly supports your problem statement about the risks of unverified AI-generated code.
Fu et al. (2023)	A Comprehensive Study on Pre-trained Models for Program Understanding and Generation.	Survey and taxonomy of pre-trained models (like CodeBERT, GraphCodeBERT) for software engineering tasks.	Provides a foundational understanding of how different LLMs are designed to comprehend code, which is essential for your LLM-based analysis agent.
Ahamd et al. (2023)	A Study on the Effectiveness of Large Language Models in Detecting Code Smells.	An empirical study comparing the ability of models like GPT-3.5 and GPT-4 to detect various code smells against traditional tools like SonarQube.	Concluded that LLMs show high precision in detecting semantic smells (like Feature Envy) but traditional tools are better at syntactic smells. This justifies your hybrid approach.
Chen et al. (2021)	Evaluating Large Language Models of Code.	Introduced the HumanEval benchmark to measure the functional correctness of code generated by LLMs.	While focused on code generation, this paper's methodology is crucial for understanding how to evaluate the quality and correctness of code, which is a core task for your 'Tester' agent.
Li et al. (2023)	Code Llama: Open Foundation Models for Code.	Introduction of Code Llama, a family of open-source LLMs specifically fine-tuned for code-related tasks.	This paper is a direct resource for selecting a potential open-source LLM for your project. It details the model's architecture and performance on various coding benchmarks.
Mastrolia et al. (2022)	On the Use of Large Language Models for Code Refactoring.	Investigated the use of LLMs to automatically suggest and perform code refactoring to remove code smells.	Demonstrates the potential for LLMs not just to detect issues but to suggest fixes, a key feature of your "actionable insights" objective.

LITERATURE SURVEY

Author(s) / Year	Title of the Paper	Methodology / Approach	Inference
Paul et al. (2024)	An Empirical Study of Security Vulnerabilities in Python Code Generated by LLMs.	Fine-tuned an LLM on a dataset of vulnerable Python code and analyzed its output for common security flaws like SQL injection and command injection.	Provides direct evidence of how and why LLMs can introduce security flaws, reinforcing the need for a post-generation audit tool like KodeHawk.
Bairi et al. (2022)	Combining Static Analysis and Transformers for Scalable and Accurate Code Analysis.	Proposed a hybrid model where a fast static analyzer first identifies potential areas of concern, which are then passed to a more powerful Transformer model for deep analysis.	This paper's methodology is almost a direct blueprint for your proposed hybrid engine, proving the validity and efficiency of the approach.
Siddiq et al. (2023)	Large Language Models for Software Engineering: A Systematic Literature Review.	A comprehensive review of how LLMs are being applied across the entire software development lifecycle, from requirements to testing.	Excellent resource for a broad overview and for positioning your project within the current research landscape. It helps justify the novelty and relevance of your work.
Hajipour et al. (2023)	A Comparative Study of Static Analysis Tools for Vulnerability Detection in Smart Contracts.	Compared multiple SAST tools (including SonarQube) for their effectiveness in finding vulnerabilities in a specific domain (smart contracts).	Although the domain is different, the methodology for comparing and evaluating SAST tools is directly applicable to your project for benchmarking your traditional analysis component.
Paul et al. (2024)	An Empirical Study of Security Vulnerabilities in Python Code Generated by LLMs.	Fine-tuned an LLM on a dataset of vulnerable Python code and analyzed its output for common security flaws like SQL injection and command injection.	Provides direct evidence of how and why LLMs can introduce security flaws, reinforcing the need for a post-generation audit tool like KodeHawk.
Bairi et al. (2022)	Combining Static Analysis and Transformers for Scalable and Accurate Code Analysis.	Proposed a hybrid model where a fast static analyzer first identifies potential areas of concern, which are then passed to a more powerful Transformer model for deep analysis.	This paper's methodology is almost a direct blueprint for your proposed hybrid engine, proving the validity and efficiency of the approach.

LITERATURE SURVEY

Author(s) / Year	Title of the Paper	Methodology / Approach	Inference
Noever (2023)	The Unreasonable Effectiveness of Multiple Agents for Software Engineering.	Explored the use of multiple LLM agents, each with a specific role (e.g., planner, coder, debugger), to solve complex software engineering tasks.	This directly supports your choice of a multi-agent architecture (LangGraph), demonstrating that a team of specialized agents outperforms a single monolithic agent.
Wang et al. (2023)	Can LLMs Patch Security Vulnerabilities?	Investigated the ability of LLMs like GPT-4 to automatically fix security vulnerabilities identified by static analysis tools.	The findings show promise but also highlight limitations where LLMs produce incorrect patches. This reinforces the scope of your project to focus on detection rather than auto-correction.
Ding et al. (2022)	An Empirical Study on the Performance of AI-based Code Generation Tools.	Analyzed the performance and resource efficiency of code generated by tools like Copilot compared to human-written code.	The study found cases where AI-generated code introduced performance bottlenecks, justifying the need for your performance analysis module.
Sobania et al. (2023)	An Analysis of the Automatic Bug Fixing Performance of ChatGPT.	Assessed ChatGPT's ability to fix bugs from the QuixBugs benchmark dataset.	Showed that while powerful, LLMs still struggle with certain classes of bugs and can introduce regressions. This highlights the unreliability of using LLMs in isolation.
Zhang et al. (2024)	From Agents to Multi-Agent Systems: A Survey of the Landscape of Agent-based AI.	A survey paper on the architectures and communication patterns used in modern multi-agent systems, including frameworks similar to LangGraph.	Provides the theoretical background for your system architecture, explaining concepts like orchestrators, specialized agents, and state management.



Thank You